Load Balancer

基本概念

- **Load Balancer (負載平衡器)** 是一種在 **多個伺服器或服務實例之間分配流量** 的元件。
- 核心目的：**避免單一伺服器過載，提升系統可用性與擴展性**。
- 對客戶端來說，它只看到一個入口，實際上流量可能被分配到多台後端伺服器。

核心功能

流量分配（Traffic Distribution）

Load Balancer 最直覺的角色就是「把流量分出去」，但真正重要的是它怎麼分，以及分流是否能跟著流量變化而調整。

在小規模系統中，流量分配可能只是單純的 Round Robin，把請求平均送到多台機器。但當流量變大、請求變複雜時，分流策略就會變成系統穩定性的關鍵。例如：

- API 請求長短差異大
- 有部分節點規格較好
- 某些機器正處於 GC 或 CPU 高峰

這時如果分配策略太天真，就會出現表面平均、實際不平均的問題。成熟的 Load Balancer 通常會結合即時負載資訊（連線數、延遲、錯誤率）做動態調整。

流量分配的本質不是「平均」，而是「讓整體系統吞吐量最大化，同時避免局部過載」。

健康檢查（Health Check）

健康檢查是 Load Balancer 真正讓系統具備自動容錯能力的核心。

它會定期對後端節點發送請求，例如：

- HTTP `/health` endpoint
- TCP port 檢查

buildmoat.org

- 或自定義的應用層檢測

如果連續多次檢查失敗，該節點就會被暫時移出流量池。當節點恢復正常後，再重新加入。

這裡的設計重點在於：

- 檢查頻率太低，故障切換慢
- 檢查頻率太高，可能造成額外負擔
- 檢查邏輯過於簡單，可能誤判（例如服務還活著但依賴的 DB 已掛）

健康檢查實際上是自動化運維的一部分，它讓「壞節點」在沒有人工介入的情況下被隔離。

高可用性（High Availability）

Load Balancer 本身也是關鍵基礎設施。如果它掛掉，整個系統就會變成單點故障。

因此在實務上，Load Balancer 通常會：

- 至少部署兩台
- 使用 failover 機制
- 搭配 VIP（Virtual IP）或雲端托管 LB 服務

在雲環境中，像託管式 Load Balancer 服務（例如雲端 L4/L7 LB）通常已內建高可用設計。但在自建架構中，必須考慮：

- LB 節點間的同步
- DNS failover
- 或使用 anycast

高可用性的本質不是「有備份」，而是故障發生時流量能否在可接受時間內自動轉移。

SSL 終結（SSL Termination）

在 HTTPS 流量中，TLS 握手與加解密會消耗 CPU。如果每台後端伺服器都處理 TLS，整體成本會放大。

SSL Termination 的做法是：

1. 在 Load Balancer 層完成 TLS 握手與解密
2. 後端只處理純 HTTP 流量

這樣的好處包括：

buildroat.org

- 減少後端 CPU 負擔
- 憑證集中管理
- 更容易統一更新 TLS 設定

不過也有設計上的考量，例如內網是否需要再次加密（mTLS）、是否有合規要求不能明文傳輸等。

在高流量系統中，SSL Termination 幾乎是標準配置。

Session Persistence（Sticky Session）

在某些系統中，使用者狀態保存在應用伺服器記憶體裡，例如購物車或臨時 session。

如果每次請求都被導向不同後端，就會導致 session 丟失。因此 Load Balancer 可以透過 cookie 或 IP hash，讓同一使用者的請求持續導向同一台機器。

這種做法可以快速解決狀態同步問題，但會帶來兩個風險：

- 流量分佈不均
- 當節點掛掉時，session 直接消失

在可擴展架構中，更理想的做法通常是把 session 外移到 Redis 或資料庫，讓應用層保持 stateless，減少對 Sticky Session 的依賴。

Sticky Session 比較像是過渡方案，而不是長期最優架構。

多區域分流（Global Load Balancing）

當系統跨多個資料中心或地理區域部署時，Load Balancer 的角色會提升到全球層級。

這時候分流依據可能包括：

- 使用者地理位置
- DNS latency
- 即時回應時間
- 區域健康狀況

常見做法包括 DNS-based routing 或 Anycast IP。

全球分流的核心目標有兩個：

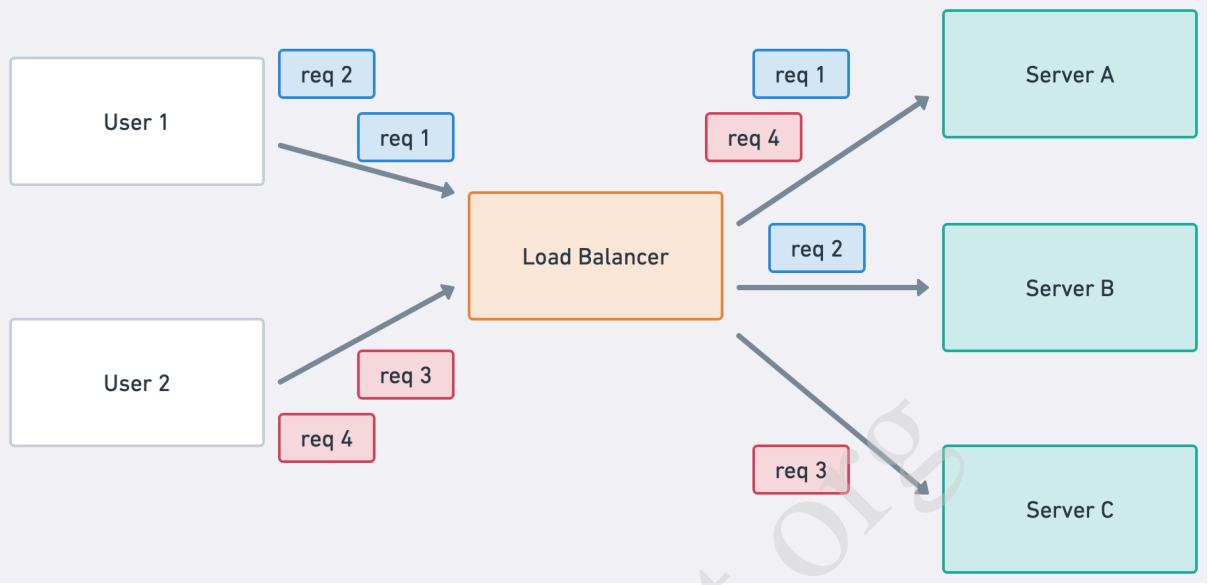
1. 降低使用者延遲
2. 當某個區域故障時，自動切換到其他區域

這層設計通常與災難復原（Disaster Recovery）策略緊密相關。

buildmoat.org

常見演算法 (Load Balancing Algorithms)

1. Round Robin



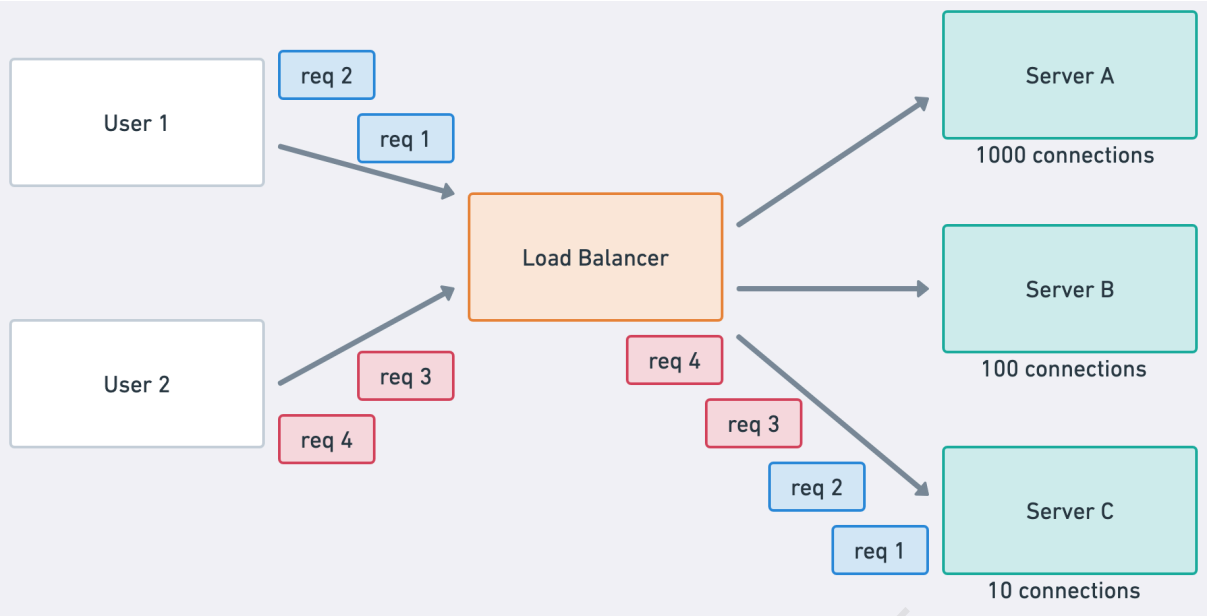
Round Robin 是最直覺、最公平的分配方式。每個新請求依序丟給下一台伺服器，不考慮當前負載狀況。它假設每個請求的成本大致相同、每台機器性能接近。

在 CPU-bound 且請求處理時間差不多的場景下，這種方法簡單有效。但如果請求時間差異很大，例如有些 API 只需 5ms，有些卻要 2 秒，Round Robin 可能會讓某些節點積壓長請求而變慢，形成實際負載不均。

它的優點是實作簡單、開銷低，適合大多數 stateless web 服務作為預設選擇。

2. Least Connections

buildroot.org

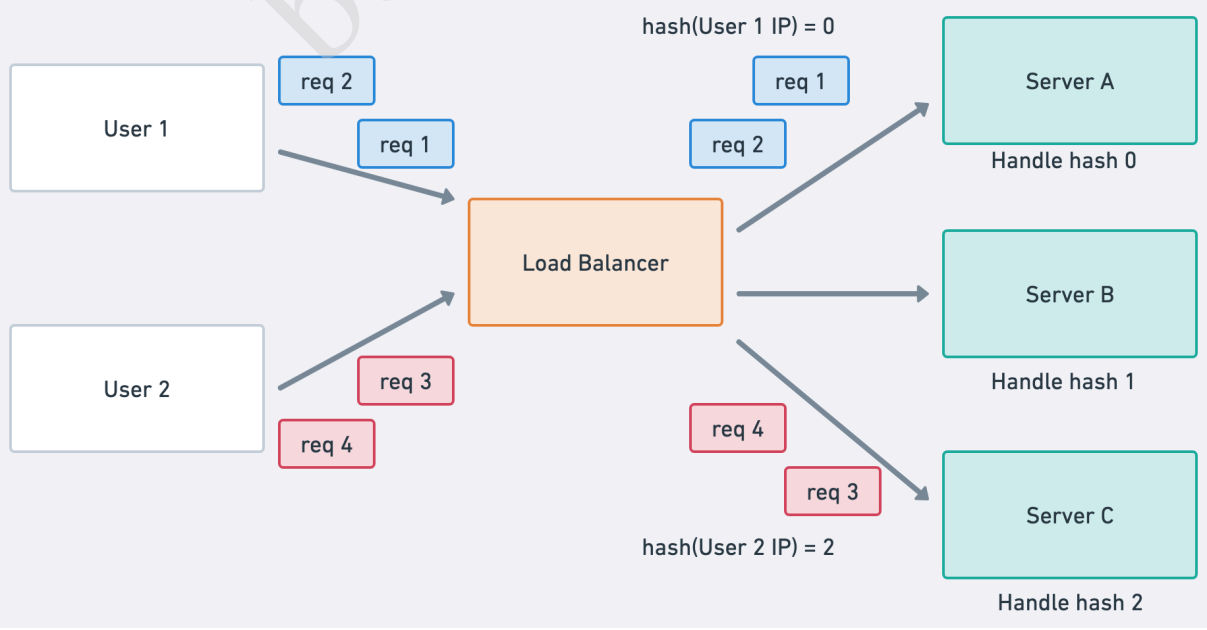


Least Connections 會把新請求送到目前「活躍連線數最少」的伺服器。它的核心假設是：連線數可以近似代表負載。

這在長連線場景（例如 WebSocket、資料庫 proxy、API 可能會 hold connection）特別有效。當不同請求處理時間差異大時，Least Connections 比 Round Robin 更能動態調整流量。

不過它也有盲點：連線數少不一定代表 CPU 使用率低。如果某台機器在做重計算但連線數不多，它仍然可能被分配更多請求。因此它比 Round Robin 聰明，但仍然不是完美的負載衡量方式。

3. IP Hash



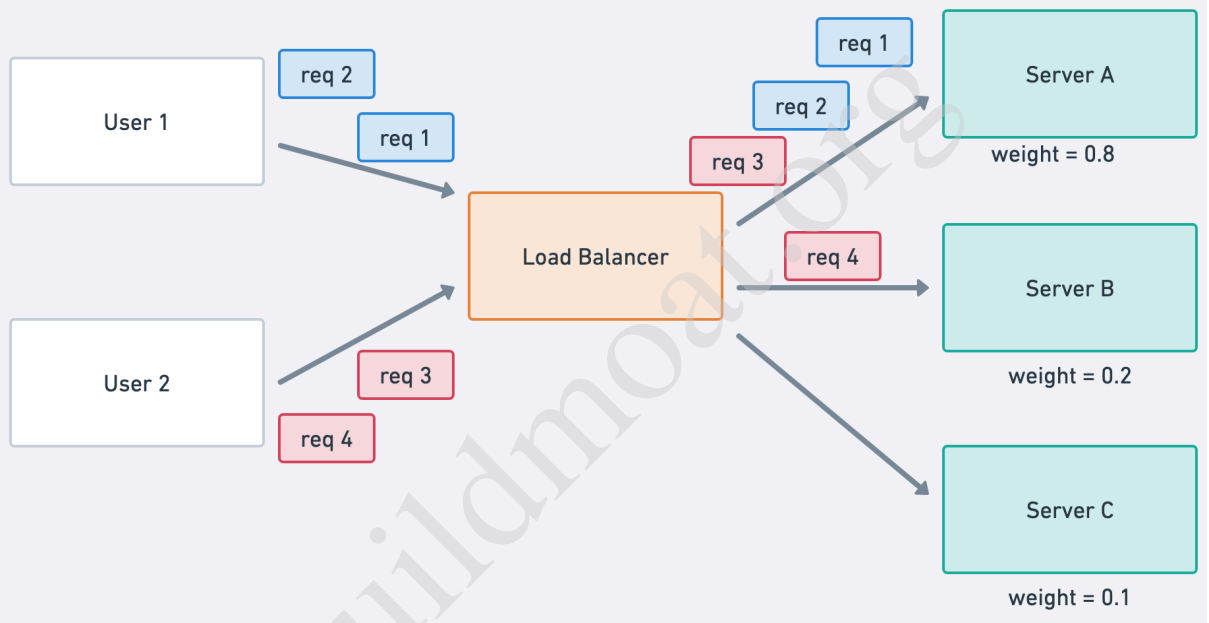
buildmoat.org

IP Hash 會對客戶端 IP 做 hash，讓同一個使用者的請求穩定導向同一台伺服器。這種策略常見於需要 session stickiness 的場景，例如伺服器記憶體中保存使用者狀態，而沒有使用外部 session store。

它的好處是簡單地實現「黏性會話」，減少跨機器同步成本。但代價是分佈可能不均。如果某些 IP（例如企業 NAT 出口）流量特別大，就會集中壓在某一台機器上。此外，當節點數量改變時，hash 結果通常會大幅變動，造成 session 重新分配。

在現代架構中，若能把 session 外移到 Redis 或 database，通常會避免依賴 IP Hash。

4. Weighted Round Robin



Weighted Round Robin 是 Round Robin 的延伸版本，允許根據伺服器性能給不同權重。性能好的機器會被分配更多請求。

這在混合規格部署（例如部分新機器、部分舊機器，或不同 CPU 核心數）時特別有用。它仍然維持 Round Robin 的簡單性，但能反映硬體差異。

需要注意的是它使用的是靜態權重，如果實際負載狀況會快速變動，僅靠權重未必能精準反映即時壓力。

問題反思：**試著從工程師的角度思考**，如果有多個 Load Balancer 同時運作，你會怎麼設計分流演算法，讓它們不需要共享狀態就能各自做出路由決策？

常見產品與服務

- **硬體型**：F5, Citrix NetScaler

buildmoat.org

- **軟體型**：NGINX, HAProxy, Envoy
- **雲端服務**：
  - AWS Elastic Load Balancing (ALB, NLB)
  - GCP Load Balancer
  - Azure Load Balancer

適用場景

- **Web / API 服務**：流量導向多台應用伺服器
- **微服務架構**：服務之間流量分配
- **多地部署**：跨區域分流，讓用戶請求到最近的資料中心
- **高可用架構**：避免單台伺服器當機導致整體系統掛掉

在系統設計面試中如何談 Load Balancer

在系統設計面試時，**不要在每個服務前面都畫一個 Load Balancer**，這樣顯得多餘，甚至會讓設計圖變得混亂。比較好的做法有兩種：

1. **乾脆省略不畫**，只要提到「這些服務是水平擴展的」就可以了。
2. **只在整個系統入口畫一個 Load Balancer**，作為抽象層，表示流量會被分配到多台服務實例。

另外，有些情境下，你需要特定的 Load Balancer 功能：

- **Sticky Sessions（黏性會話）** → 確保同一用戶的請求都導向同一伺服器，例如購物車
- **Persistent Connections（長連線）** → 像 WebSockets，需要連線固定在同一後端

在面試中，最常見的判斷是：**要用 L4 還是 L7 的 Load Balancer？**

- **L4 (Transport Layer)** → 基於 TCP/UDP，適合長連線（如 WebSocket），因為它只管傳輸，不會頻繁斷開重建
- **L7 (Application Layer)** → 基於 HTTP/HTTPS，能根據 URL、Header、Method 做更靈活的流量路由，也能減少下游的連線負擔

速答法則：

buildfast.org

- 如果有 **WebSocket 或持久連線** → 用 **L4 LB**
- 如果是 **一般 Web 流量** → 用 **L7 LB**，因為它提供更靈活的路由

buildmoat.org